

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

Implémentation et administration d'une base de données (SGBD, SQL)

Module 3 : Systèmes d'information et bases de données

Consigne : Les exercices suivants portent sur la conception, l'implémentation et l'administration d'une base de données relationnelle sous MySQL. Toute réponse doit être argumentée ; les requêtes SQL doivent être syntaxiquement correctes et optimisées.

EXERCICE 1 Modélisation avancée et DDL contraignant

Une plateforme de e-learning gère des Apprenants, des Cours, des Sessions (début, fin, formateur) et des Inscriptions (note, statut). Un apprenant s'inscrit à une session d'un cours. Un formateur anime plusieurs sessions. Un cours peut avoir plusieurs sessions. Les règles de gestion imposent : (1) L'email de l'apprenant est unique. (2) La note est entre 0 et 20 inclus, ou NULL si non évalué. (3) Le statut d'inscription vaut 'EN_COURS', 'TERMINE', 'ABANDON' (défaut 'EN_COURS'). (4) Une session ne peut excéder 30 apprenants. (5) Supprimer un cours cascade sur ses sessions.

1. Proposer le MCD complet (entités, associations, cardinalités, identifiants relatifs). Justifier le choix des identifiants pour *Inscription*.
2. Écrire le script SQL (CREATE TABLE) implémentant ce schéma avec toutes les contraintes d'intégrité (PK, FK, UNIQUE, CHECK, DEFAULT, NOT NULL). Utiliser InnoDB.
3. La règle (4) ne peut être assurée par une contrainte standard. Proposer deux stratégies techniques distinctes pour l'implémenter (ex : trigger, procédure stockée, contrôle applicatif). Comparer leurs avantages/inconvénients en termes d'atomicité et de performance.
4. Écrire l'ordre ALTER TABLE pour ajouter une colonne date_inscription (DATETIME, défaut CURRENT_TIMESTAMP) à *Inscription*, puis créer un index composite sur (id_session, statut).

EXERCICE 2 Requêtes complexes, agrégation et jointures externes

On considère la base implémentée à l'exercice 1, peuplée de données significatives.

1. Lister les apprenants (nom, email) n'ayant jamais été inscrits à aucune session (jointure externe ou opérateur ensembliste).
2. Pour chaque cours, afficher son titre, le nombre total de sessions ouvertes, le nombre total d'inscrits toutes sessions confondues, et le taux de remplissage global (inscrits / (sessions × 30)) arrondi à 2 décimales. Trier par taux décroissant.

3. Identifier les sessions où la moyenne des notes des apprenants ayant le statut 'TERMINE' est strictement supérieure à la moyenne globale de toutes les notes 'TERMINE' de la base. Afficher `id_session`, `date_debut`, `moyenne_session`, `moyenne_globale`.
4. Écrire une requête paramétrée (préparée) pour mettre à jour le statut des inscriptions d'une session donnée (?) à 'TERMINE' uniquement pour les apprenants ayant une note ≥ 10 , et à 'ECHEC' pour les autres (note < 10 ou NULL). Gérer le cas où la session n'existe pas.

EXERCICE 3 Administration, sauvegarde et évolution de schéma

La base `elearning_db` (500 Mo, 2M lignes dans `Inscription`) est en production 24/7.

1. Comparer `mysqldump --single-transaction --quick` et une sauvegarde physique (copie fichiers `/var/lib/mysql/` après `FLUSH TABLES WITH READ LOCK`). Citer 3 critères : cohérence transactionnelle, temps de verrouillage, portabilité inter-versions.
2. La direction exige d'historiser les changements de statut d'inscription (ancien, nouveau, date, utilisateur). Proposer une table `Historique_Statut` et le TRIGGER AFTER UPDATE sur `Inscription` pour l'alimenter. Gérer le cas où le statut ne change pas.
3. L'index composite (`id_session`, `statut`) créé en Ex.1.4 ralentit les insertions massives nocturnes. Proposer une stratégie de maintenance (désactivation/réactivation, reconstruction) sans indisponibilité prolongée. Commandes SQL exactes.
4. Analyser l'impact du passage en `utf8mb4` (depuis `latin1`) sur le stockage des index `VARCHAR(255)` (ex : `email`). Calculer la taille max d'index InnoDB (767 octets ou 3072 octets selon `innodb_large_prefix`) et proposer une solution si la limite est dépassée.

EXERCICE 4 Étude de cas : Optimisation et sécurité

Un audit révèle une requête lente : `SELECT * FROM Inscription i JOIN Session s ON i.id_session=s.id WHERE s.date_debut BETWEEN '2024-01-01' AND '2024-12-31' AND i.statut='TERMINE' ORDER BY i.note DESC LIMIT 20;`

1. Expliquer pourquoi `SELECT *` est pénalisant ici (I/O, buffer pool, index covering). Ré-écrire la requête en ne sélectionnant que les colonnes nécessaires pour un index couvrant optimal.
2. Proposer la définition exacte de l'index composite optimal pour cette requête (ordre des colonnes, sens ASC/DESC). Justifier l'ordre par la sélectivité et l'élimination du `filesort`.
3. La base contient des données sensibles (emails, notes). Décrire la mise en œuvre du chiffrement au repos (TDE) et en transit (TLS) sous MySQL 8.0. Citer les variables sys-

tème à vérifier.

4. Un développeur propose d'ajouter une colonne `json_metadata` (JSON) dans `Apprenant` pour stocker des données hétérogènes (adresses, préférences). Critiquer ce choix vs normalisation (1NF, 3NF) : performances recherche, validation schéma, indexation, migration future.